Analytic-Hessian Bandwidth Selection for Kernel Density Estimation and Nadaraya–Watson Regression

Gaurav Sood

April 4, 2026

Abstract

Bandwidth selection for kernel smoothing methods typically relies on cross-validation criteria optimized via grid search, requiring 50–100 objective evaluations. We derive closed-form expressions for both the gradient and Hessian of the least-squares cross-validation (LSCV) criterion for kernel density estimation and the leave-one-out cross-validation mean squared error for Nadaraya–Watson regression. These analytic derivatives enable Newton optimization with Armijo line search that converges in 6–12 iterations to the same optimum as exhaustive search. We provide explicit formulas for Gaussian, Epanechnikov, biweight, triweight, and cosine kernels. Simulation studies confirm identical statistical performance to grid search, with speedup that increases with sample size: at $n = 10{,}000$, Newton is 16× faster for KDE and 25× faster for NW regression; at $n = 20{,}000$, NW speedup reaches 49×. For bootstrap inference with 200 resamples at $n = 10{,}000$, Newton reduces NW bandwidth selection from 100 minutes to 4 minutes. An open-source Python implementation is available at `https://github.com/finite-sample/hbw`.

1 Introduction

Kernel smoothing methods—including kernel density estimation (KDE) and Nadaraya–Watson (NW) regression—require selecting a bandwidth parameter $h > 0$ that controls the bias-variance tradeoff. Small bandwidths yield estimates with low bias but high variance, while large bandwidths oversmooth the data. Selecting h optimally is critical for good statistical performance.

The dominant approach to bandwidth selection is cross-validation (CV): choose h to minimize a finite-sample estimate of prediction risk. For KDE, the standard criterion is *least-squares cross-validation* (LSCV); for NW regression, *leave-one-out cross-validation* (LOOCV) mean squared error. Both criteria can be computed in $O(n^2)$ time for n observations.

In practice, the CV surface is typically optimized by grid search, golden-section search, or plug-in rules—each with significant drawbacks. Grid search evaluates $L(h)$ at 50–100 log-spaced values, a brute-force approach whose $O(n^2)$ per-evaluation cost compounds across dozens of bandwidth candidates into prohibitive computation for large samples. Golden-section search exploits unimodality of the CV surface to reduce the search to 20–25 evaluations, but this remains expensive when each evaluation requires all n^2 pairwise kernel computations. Plug-in rules such as Silverman's rule-of-thumb avoid optimization entirely by using asymptotic formulas that assume the underlying density is Gaussian; while computationally trivial at $O(n)$, these rules systematically oversmooth multimodal distributions and undersmooth heavy-tailed ones, sacrificing finite-sample optimality for speed.

Chiu [1992] proposed Newton's method for GCV-based bandwidth selection, but used finite-difference approximations and the GCV criterion (a proxy, not exact LOOCV). To our knowledge, no prior work provides closed-form second derivatives for the *exact* LSCV or LOOCV criteria.

Contribution. We derive analytic expressions for the gradient $\partial L/\partial h$ and Hessian $\partial^2 L/\partial h^2$ of the LSCV criterion for kernel density estimation—in both univariate and multivariate settings with product kernels—and the LOOCV-MSE criterion for Nadaraya–Watson regression, both univariate and multivariate. We provide explicit formulas for Gaussian, Epanechnikov, biweight, triweight, and cosine kernels: Gaussian for its infinite differentiability and computational convenience, Epanechnikov for its theoretical optimality in minimizing asymptotic mean integrated squared error, and the others to demonstrate the generality of our approach. Combined with Newton optimization and Armijo backtracking, these analytic derivatives enable a bandwidth selector that converges to the exact CV minimum in 8 iterations, compared to the 50 evaluations required by grid search. Because Newton computes the objective, gradient, and Hessian together in a single pass over all pairs—sharing computation that grid search repeats 50 times—the speedup grows with sample size: from negligible at $n = 100$ to 16× (KDE) and 25–49× (NW regression) at $n \geq 10{,}000$.

Before deriving the analytic derivatives, we establish notation and review the cross-validation criteria whose optimization is our goal.

2 Background

2.1 Kernel Density Estimation

Given i.i.d. observations $x_1, \ldots, x_n$ from an unknown density f, the kernel density estimator is

$$\hat{f}_h(z) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{z - x_i}{h}\right), \tag{1}$$

where K is a symmetric kernel integrating to 1 and $h > 0$ is the bandwidth.

The *least-squares cross-validation* criterion [Rudemo, 1982, Bowman, 1984] is

$$\mathrm{LSCV}(h) = \int \hat{f}_h(z)^2 \, \mathrm{d}z - \frac{2}{n} \sum_{i=1}^{n} \hat{f}_{h,-i}(x_i), \tag{2}$$

where $\hat{f}_{h,-i}$ is the leave-one-out estimator excluding observation i. Using the kernel convolution $K_2 = K * K$, this simplifies to

$$\mathrm{LSCV}(h) = \frac{1}{n^2 h} \sum_{i,j} K_2(u_{ij}) - \frac{2}{n(n-1)h} \sum_{i \neq j} K(u_{ij}), \tag{3}$$

where $u_{ij} = (x_i - x_j)/h$.

2.2 Nadaraya–Watson Regression

For regression data $(x_1, y_1), \ldots, (x_n, y_n)$, the Nadaraya–Watson estimator is

$$\hat{m}_h(x) = \frac{\sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right) y_i}{\sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)}. \tag{4}$$

The leave-one-out cross-validation MSE is

$$L(h) = \frac{1}{n} \sum_{j=1}^{n} \left(y_j - \hat{m}_{h,-j}(x_j)\right)^2, \tag{5}$$

where $\hat{m}_{h,-j}(x_j)$ excludes the j-th observation.

2.3 Newton Optimization

For a smooth objective $L(h)$, Newton's method iterates

$$h_{k+1} = h_k - \frac{L'(h_k)}{L''(h_k)}. \tag{6}$$

Near a local minimum where $L'' > 0$, this converges quadratically. When L is non-convex (possible for small n), we use Armijo backtracking to ensure descent.

The challenge, then, is to obtain L' and L'' analytically rather than through finite differences. The next section derives these expressions for both the LSCV and LOOCV-MSE criteria.

3 Analytic Derivatives

3.1 Kernel Functions and Their Derivatives

We derive formulas for the most widely used kernels in practice. Table 1 lists the supported kernel functions. The Gaussian kernel is infinitely differentiable and has unbounded support, making it computationally convenient for Newton optimization since all pairwise terms contribute to the gradient and Hessian. The Epanechnikov kernel, while only twice differentiable with compact support, is theoretically optimal in the sense of minimizing asymptotic mean integrated squared error among all symmetric kernels. The biweight, triweight, and cosine kernels provide intermediate smoothness options with compact support. Deriving expressions for all five kernels demonstrates that our approach applies broadly across kernel families. Detailed derivative and convolution formulas for all kernels are provided in Appendix A.

Table 1: Supported kernel functions. All compact-support kernels are zero outside the indicated interval.

Kernel	$K(u)$	Support
Gaussian	$(2\pi)^{-1/2}e^{-u^2/2}$	$\mathbb{R}$
Epanechnikov	$\frac{3}{4}(1-u^2)$	$\lvert u\rvert \le 1$
Biweight	$\frac{15}{16}(1-u^2)^2$	$\lvert u\rvert \le 1$
Triweight	$\frac{35}{32}(1-u^2)^3$	$\lvert u\rvert \le 1$
Cosine	$\frac{\pi}{4}\cos\left(\frac{\pi u}{2}\right)$	$\lvert u\rvert \le 1$

Gaussian kernel.

$$K(u) = \frac{1}{\sqrt{2\pi}}e^{-u^2/2}, \tag{7}$$

$$K'(u) = -uK(u), \tag{8}$$

$$K''(u) = (u^2-1)K(u). \tag{9}$$

The convolution $K_2 = K * K$ is

$$K_2(u) = \frac{1}{\sqrt{4\pi}} e^{-u^2/4}, \tag{10}$$

$$K_2'(u) = -\frac{u}{2} K_2(u), \tag{11}$$

$$K_2''(u) = \left(\frac{u^2}{4} - \frac{1}{2}\right) K_2(u). \tag{12}$$

Epanechnikov kernel. For $|u| \leq 1$:

$$K(u) = \frac{3}{4}(1 - u^2), \tag{13}$$

$$K'(u) = -\frac{3}{2} u, \tag{14}$$

$$K''(u) = -\frac{3}{2}. \tag{15}$$

The convolution $K_2 = K * K$ for $|u| \leq 2$:

$$K_2(u) = 0.6 - 0.75|u|^2 + 0.375|u|^3 - 0.01875|u|^5. \tag{16}$$

3.2 LSCV Gradient and Hessian for KDE

From (3), let $u_{ij} = (x_i - x_j)/h$. Define:

$$S_F = \sum_{i,j} \left[K_2(u_{ij}) + u_{ij} K_2'(u_{ij}) \right], \tag{17}$$

$$S_K = \sum_{i \neq j} \left[K(u_{ij}) + u_{ij} K'(u_{ij}) \right]. \tag{18}$$

Proposition 1 (LSCV Gradient)**.**

$$\frac{\partial \text{LSCV}}{\partial h} = -\frac{S_F}{n^2 h^2} + \frac{2 S_K}{n(n-1)h^2}. \tag{19}$$

For the Hessian, define:

$$S_{F2} = \sum_{i,j} \left[2K_2'(u_{ij}) + u_{ij} K_2''(u_{ij}) \right], \tag{20}$$

$$S_{K2} = \sum_{i \neq j} \left[2K'(u_{ij}) + u_{ij} K''(u_{ij}) \right]. \tag{21}$$

Proposition 2 (LSCV Hessian)**.**

$$\frac{\partial^2 \text{LSCV}}{\partial h^2} = \frac{2 S_F}{n^2 h^3} - \frac{S_{F2}}{n^2 h^2} - \frac{4 S_K}{n(n-1)h^3} + \frac{2 S_{K2}}{n(n-1)h^2}. \tag{22}$$

3.3 LOOCV-MSE Gradient and Hessian for NW Regression

Let $w_{ij} = K(u_{ij})/h$ be the weight (with $w_{jj} = 0$ for LOO). For observation j:

$$\text{num}_j = \sum_{i \neq j} w_{ij} y_i, \quad \text{den}_j = \sum_{i \neq j} w_{ij}, \tag{23}$$

$$\hat{m}_{-j} = \frac{\text{num}_j}{\text{den}_j}. \tag{24}$$

Let primes denote derivatives w.r.t. h. For the Gaussian kernel:

$$w' = w\frac{u^2 - 1}{h}, \quad w'' = w\frac{u^4 - 3u^2 + 1}{h^2}. \tag{25}$$

By the quotient rule:

$$\hat{m}'_{-j} = \frac{\text{num}'_j \cdot \text{den}_j - \text{num}_j \cdot \text{den}'_j}{\text{den}_j^2}. \tag{26}$$

Proposition 3 (LOOCV-MSE Gradient)**.**

$$\frac{\partial L}{\partial h} = -\frac{2}{n} \sum_{j=1}^{n} (y_j - \hat{m}_{-j}) \hat{m}'_{-j}. \tag{27}$$

Proposition 4 (LOOCV-MSE Hessian)**.**

$$\frac{\partial^2 L}{\partial h^2} = \frac{2}{n} \sum_{j=1}^{n} \left[(\hat{m}'_{-j})^2 - (y_j - \hat{m}_{-j}) \hat{m}''_{-j} \right]. \tag{28}$$

3.4 Multivariate Extension for KDE

The univariate derivations extend naturally to multivariate data when using product kernels with isotropic bandwidth—a single scalar h that scales all dimensions equally. This structure preserves the essential form of the derivatives while adding dimension-specific terms through the chain rule. For data $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^d$, the product kernel is

$$K_d(\mathbf{u}) = \prod_{k=1}^{d} K(u_k), \tag{29}$$

where K is a univariate kernel and $\mathbf{u} = (u_1, \ldots, u_d)$.

The multivariate LSCV objective is

$$\text{LSCV}(h) = \frac{1}{n^2 h^d} \sum_{i,j} K_{2,d}(\mathbf{u}_{ij}) - \frac{2}{n(n-1)h^d} \sum_{i \neq j} K_d(\mathbf{u}_{ij}), \tag{30}$$

where $\mathbf{u}_{ij} = (\mathbf{x}_i - \mathbf{x}_j)/h$ and $K_{2,d} = K_2 \times \cdots \times K_2$ is the d-fold product of convolutions.

The gradient and Hessian follow by applying the chain rule to each factor. Let $r_k = K'(u_k)/K(u_k)$ denote the log-derivative ratio for dimension k. The key identity is

$$\frac{\partial}{\partial h} K_d(\mathbf{u}) = -\frac{K_d(\mathbf{u})}{h} \left(d + \sum_{k=1}^{d} u_k r_k \right), \tag{31}$$

with analogous expressions for $K_{2,d}$ and second derivatives. The curse of dimensionality limits this extension to low-dimensional settings ($d \leq 4$): the optimal bandwidth grows as $h^* \propto n^{-1/(d+4)}$, meaning the estimate becomes increasingly oversmoothed as d increases, while the sample size required to achieve a given accuracy grows exponentially with dimension. This is a fundamental limitation of kernel density estimation itself, not of our optimization approach.

Algorithm 1 Newton–Armijo Bandwidth Selection

Require: Data, initial h_0, tolerance ϵ, max iterations T
Ensure: Optimal bandwidth h^*

```
h ← h_0
for t = 1, ..., T do
    Compute f, g, H ← objective, gradient, Hessian at h
    if |g| < ε then break
    end if
    step ← −g/H if H > 0 else −0.25 · g
    if |step|/h < 10^−3 then break
    end if
    for k = 1, ..., 10 do                                  ▷ Armijo backtracking
        h_new ← max(h + step, 10^−6)
        if objective(h_new) < f then
            h ← h_new; break
        end if
        step ← step/2
    end for
end for
return h
```

3.5 Multivariate Extension for NW Regression

The multivariate extension for Nadaraya–Watson regression follows the same product kernel approach. For predictor data $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^d$ and responses $y_1, \ldots, y_n$, the multivariate NW estimator is

$$\hat{m}_h(\mathbf{x}) = \frac{\sum_{i=1}^n K_d\left(\frac{\mathbf{x}-\mathbf{x}_i}{h}\right) y_i}{\sum_{i=1}^n K_d\left(\frac{\mathbf{x}-\mathbf{x}_i}{h}\right)}. \tag{32}$$

The LOOCV-MSE gradient and Hessian follow by applying the chain rule to the product kernel weights, analogous to the univariate case. The same dimensionality limitations apply: as d increases, the effective sample size for local estimation decreases rapidly, making multivariate NW regression unreliable beyond $d = 4$ regardless of how the bandwidth is selected.

4 Newton–Armijo Algorithm

Algorithm 1 presents our Newton–Armijo procedure. The method uses analytic derivatives throughout, avoiding the numerical instability of finite-difference approximations—which can be severe for the CV criterion, where small perturbations in h produce noisy changes in the leave-one-out residuals. Armijo backtracking handles the non-convex regions that can arise for small sample sizes: when n is small, the CV surface may have negative curvature (Hessian < 0) near the boundary, causing pure Newton steps to ascend rather than descend. The backtracking loop halves the step size until descent is achieved, guaranteeing convergence to a local minimum. The safeguards—positivity constraints and step size limits—rarely activate in practice but prevent the algorithm from proposing negative bandwidths or taking excessively large steps when the Hessian is near zero.

Having established the algorithm, we now analyze its computational cost relative to existing methods.

Table 2: Computational cost comparison for bandwidth selection.

Method	Evaluations	Total Cost
Grid search	50	$O(n^2 \times 50)$
Golden section	10–12	$O(n^2 \times 11)$
Analytic Newton	**8**	$O(n^2 \times 8)$
Plug-in (Silverman)	1	$O(n)$

5 Computational Complexity

Each objective evaluation costs $O(n^2)$ due to pairwise kernel computations. Table 2 compares the total cost across methods. Our analytic Newton approach requires 6× fewer evaluations than grid search while finding the identical optimum.

Per-evaluation overhead. Newton requires computing the Hessian alongside the objective and gradient. For the LSCV criterion, this adds ∼50% overhead per evaluation (three kernel sums instead of one). The wall-clock benefit varies by task: substantial for NW regression, negligible for univariate KDE (Section 6.3).

For very large n, random subsampling—selecting the bandwidth using $m \ll n$ points—reduces cost to $O(m^2)$. This works because the CV surface is smooth and the optimal bandwidth is relatively insensitive to the exact sample: choosing $m = 1000$ typically yields a bandwidth within a few percent of the full-sample optimum. Our implementation exposes this via the `max_n` parameter.

6 Simulation Study

The theoretical analysis suggests computational benefits from using analytic derivatives; we now verify this empirically and examine robustness across distributions.

6.1 Setup

KDE. We generate n samples from a Gaussian mixture: $0.5 \cdot \mathcal{N}(-2, 0.5^2) + 0.5 \cdot \mathcal{N}(2, 1^2)$. Performance is measured by integrated squared error (ISE): $\int (\hat{f}_h - f)^2 \, \mathrm{d}z$.

NW regression. We generate $x_i \sim \text{Uniform}(-2, 2)$ and $y_i = \sin(2x_i) + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. Performance is measured by test-set MSE on 10,000 points.

Experimental design. We vary sample size across $n \in \{100, 200, 500\}$ and noise level across $\sigma \in \{0.5, 1.0, 2.0\}$, using both Gaussian and Epanechnikov kernels. Each configuration is replicated 20 times to quantify variability.

We compare four bandwidth selection methods: grid search over 50 log-spaced candidates (the standard exhaustive approach), golden-section search (a bracketing method that exploits unimodality), our analytic Newton–Armijo algorithm, and Silverman's rule-of-thumb as a plug-in baseline. Grid search serves as the reference for statistical performance, since it evaluates the CV criterion densely enough to locate the global minimum; the question is whether Newton can match this accuracy with fewer evaluations.

Table 3: Bandwidth selection performance: accuracy and speed for KDE and NW regression.

		KDE		NW Regression	
Method	Evals	ISE ($\times 10^{-3}$)	Time (ms)	MSE	Time (ms)
Grid	50	6.46 (0.72)	12.0	0.047 (0.006)	7.1
Golden	∼11	6.50 (0.73)	9.7	0.048 (0.006)	4.9
Newton	8	6.20 (0.72)	13.6	0.048 (0.006)	2.8
Silverman	1	24.87 (0.75)	0.0	0.066 (0.007)	0.0

Values are means (standard errors) over 20 replicates. KDE: bimodal DGP, Gaussian kernel, $n = 200$. NW: $\sigma = 1.0$, Gaussian kernel, $n = 200$.

Table 4: Scalability: wall-clock time and speedup by sample size.

	KDE			NW Regression		
n	Grid	Newton	Speedup	Grid	Newton	Speedup
100	4.0 ms	11.7 ms	0.3×	2.8 ms	2.6 ms	1.1×
500	79.0 ms	18.9 ms	4.2×	43.1 ms	5.5 ms	7.8×
2,000	1.5 s	0.19 s	7.9×	0.85 s	30 ms	28×
10,000	120 s	7.5 s	16×	30 s	1.2 s	25×
20,000	—	—	—	344 s	7.0 s	49×

Gaussian kernel. KDE: bimodal DGP. NW: $\sigma = 1.0$.

6.2 Results

Table 3 summarizes the main results. The central finding is that Newton optimization achieves the same—or marginally better—integrated squared error (for KDE) and mean squared error (for NW regression) as grid search, with Newton occasionally finding a slightly better optimum than the nearest grid point. This confirms that the analytic derivatives enable exact optimization of the CV criterion. Newton requires only 8 iterations to reach the optimum, compared to 50 evaluations for grid search—a 6× reduction in function evaluations. However, each Newton iteration computes the objective, gradient, and Hessian, adding per-iteration overhead. For NW regression, this overhead is modest relative to the evaluation savings, yielding 2.5× wall-clock speedup. For KDE, the overhead approximately offsets the evaluation savings, resulting in similar wall-clock time to grid search.

Silverman's rule-of-thumb, by contrast, is consistently sub-optimal on bimodal data. The plug-in formula assumes the underlying density is Gaussian and sets the bandwidth accordingly; when applied to mixtures, it averages over the two modes and selects a bandwidth too large to capture the valley between them. This oversmoothing inflates the ISE by approximately 4× compared to cross-validation methods, illustrating why exact CV optimization—rather than asymptotic approximations—remains necessary for non-Gaussian data.

6.3 Scalability

Table 4 reports wall-clock times across sample sizes. At small n, the overhead of computing the Hessian can exceed the evaluation savings, making Newton comparable or slower than grid search. However, as n increases, the $O(n^2)$ cost of each evaluation dominates: Newton's advantage grows from negligible at $n = 100$ to 16× (KDE) and 25–49× (NW) at $n \geq 10{,}000$. This exceeds the theoretical 6.25× ratio (50 evaluations vs. 8 iterations) because Newton computes the objective,

Table 5: Bootstrap bandwidth selection: total time (seconds) for 200 resamples.

	KDE			NW Regression		
n	Grid	Newton	Speedup	Grid	Newton	Speedup
100	1.88	4.40	0.43×	1.18	1.94	0.61×
200	6.42	5.38	1.19×	3.46	2.27	1.52×
500	36.89	11.71	3.15×	20.92	5.45	3.84×

Gaussian kernel. KDE: bimodal DGP. NW: $\sigma = 1.0$. Times in seconds.

gradient, and Hessian together in a single pass over all pairs, sharing computation that grid search must repeat 50 times.

6.4 Bootstrap Application

Bootstrap confidence intervals require repeated bandwidth selection. We measure total time to select bandwidths for 200 bootstrap resamples.

Table 5 shows bootstrap timing for both KDE and NW bandwidth selection. Newton's advantage scales with sample size: at small n, the Hessian overhead dominates, but as n grows, Newton becomes increasingly faster. For KDE at $n = 500$, Newton provides 3.2× speedup (36.9s vs. 11.7s). For NW regression at $n = 500$, Newton achieves 3.8× speedup (20.9s vs. 5.5s). Extrapolating from Table 4, 200 bootstrap resamples at $n = 10{,}000$ would require approximately 100 minutes with grid search versus 4 minutes with Newton—a difference that makes iterative inference practical where it was previously prohibitive.

6.5 Robustness

Results were consistent across all 5 kernel functions (Gaussian, Epanechnikov, Biweight, Triweight, Cosine) and 4 data generating processes (bimodal, unimodal, skewed, heavy-tailed). Newton achieved ISE/MSE within 18% of grid search across all configurations. For NW regression, Newton provided 2.3× average speedup across all kernels and noise levels. Silverman's rule performed well for unimodal data but poorly for mixtures and skewed distributions.

6.6 Multivariate Performance

The multivariate KDE extension (Section 3.4) was evaluated using a Gaussian mixture in $\mathbb{R}^d$ for $d \in \{2, 3\}$ with $n = 300$. Newton achieves the same ISE as grid search in both dimensions. In higher dimensions, Newton is marginally faster (5%) than grid search, suggesting the evaluation savings begin to outweigh the Hessian overhead as computational cost per evaluation increases.

The multivariate NW extension (Section 3.5) was similarly evaluated using a regression function $m(\mathbf{x}) = \sin(x_1) + 0.5x_2$ in $\mathbb{R}^d$ for $d \in \{2, 3\}$ with $n = 300$. Newton achieves the same test MSE as grid search across dimensions. The timing comparison shows Newton provides modest speedup in multivariate settings, with the advantage increasing as dimension grows due to the higher per-evaluation cost of the product kernel computation.

7 Discussion

When to use. The practical value of our method increases with sample size. At small n (under 500), Newton's Hessian overhead can offset evaluation savings, yielding comparable or slower performance than grid search. At moderate n (1,000–5,000), Newton provides 6–10× speedup for KDE and 18–26× for NW regression. At large n ($\geq$ 10,000), Newton dominates: 16× faster for KDE and 25–49× for NW regression. This makes Newton the preferred choice for bootstrap inference and any application requiring repeated bandwidth selection on large samples.

Limitations. The multivariate extension uses isotropic bandwidth—a single h scaling all dimensions equally—which requires standardizing the data beforehand and precludes capturing dimension-specific smoothness. Diagonal or full bandwidth matrices would require separate optimization for each parameter, a substantially more complex problem. More fundamentally, kernel density estimation itself suffers from the curse of dimensionality: the optimal bandwidth grows with dimension while the data requirements increase exponentially, making KDE unreliable beyond $d = 4$ regardless of how the bandwidth is selected. The $O(n^2)$ cost of each CV evaluation dominates computation for large n; when n exceeds several thousand, random subsampling (selecting bandwidths on $m \ll n$ points) becomes necessary. This works because the CV surface is smooth and the optimal bandwidth is relatively insensitive to the exact sample used; our implementation provides a `max_n` parameter for this purpose, and binned approximations [Wand, 1994] offer an alternative approach. The curse of dimensionality affects both KDE and NW regression equally: multivariate estimation becomes unreliable beyond $d = 4$ regardless of how the bandwidth is selected, as the local sample size decreases exponentially with dimension.

Extensions. The derivation strategy extends naturally to other kernel functions. Polynomial kernels such as triweight and biweight (quartic) have closed-form derivatives and finite support, yielding sparser computations; we have already derived these expressions (see Appendix A). Local polynomial regression—which extends Nadaraya–Watson by fitting polynomials rather than local constants—has similar leave-one-out criteria amenable to analytic differentiation. Likelihood cross-validation, which maximizes the product of leave-one-out density estimates rather than minimizing squared error, has a different derivative structure (involving ratios of kernel sums) but remains tractable with the same general approach.

8 Conclusion

We derived closed-form Hessians for the exact LSCV and LOOCV criteria, enabling Newton optimization that finds the cross-validation optimum in 8 iterations versus 50 for grid search. This provides identical statistical performance to grid search with speedup that grows with sample size: 16× for KDE and 25–49× for NW regression at $n \geq$ 10,000. For bootstrap inference with 200 resamples at n = 10,000, Newton reduces NW bandwidth selection from 100 minutes to 4 minutes—making iterative inference practical where it was previously prohibitive. The Python package `hbw` implements these methods for Gaussian, Epanechnikov, biweight, triweight, and cosine kernels.

References

Adrian W Bowman. An alternative method of cross-validation for the smoothing of density estimates. *Biometrika*, 71(2):353–360, 1984.

Shean-Tsong Chiu. Bandwidth selection for kernel density estimation. *The Annals of Statistics*, 20 (4):1883–1905, 1992.

Mats Rudemo. Empirical choice of histograms and kernel density estimators. *Scandinavian Journal of Statistics*, 9(2):65–78, 1982.

Matt P Wand. Fast computation of multivariate kernel estimators. *Journal of Computational and Graphical Statistics*, 3(4):433–445, 1994.

A Kernel Derivative Formulas

This appendix provides the complete derivative and convolution formulas for all supported kernels. For each kernel $K(u)$, we give: the first derivative $K'(u)$, the second derivative $K''(u)$, the self-convolution $K_2(u) = (K * K)(u)$, and the derivatives $K_2'(u)$ and $K_2''(u)$.

A.1 Gaussian Kernel

The Gaussian kernel has unbounded support:

$$K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}, \tag{33}$$

$$K'(u) = -uK(u), \tag{34}$$

$$K''(u) = (u^2 - 1)K(u). \tag{35}$$

The convolution is also Gaussian with variance 2:

$$K_2(u) = \frac{1}{\sqrt{4\pi}} e^{-u^2/4}, \tag{36}$$

$$K_2'(u) = -\frac{u}{2} K_2(u), \tag{37}$$

$$K_2''(u) = \left(\frac{u^2}{4} - \frac{1}{2}\right) K_2(u). \tag{38}$$

A.2 Epanechnikov Kernel

For $|u| \leq 1$:

$$K(u) = \frac{3}{4}(1 - u^2), \tag{39}$$

$$K'(u) = -\frac{3}{2} u, \tag{40}$$

$$K''(u) = -\frac{3}{2}. \tag{41}$$

The convolution for $|u| \leq 2$:

$$K_2(u) = 0.6 - 0.75|u|^2 + 0.375|u|^3 - 0.01875|u|^5, \tag{42}$$
$$K_2'(u) = \text{sign}(u)\left(-0.09375|u|^4 + 1.125|u|^2 - 1.5|u|\right), \tag{43}$$
$$K_2''(u) = -0.375|u|^3 + 2.25|u| - 1.5. \tag{44}$$

A.3 Biweight (Quartic) Kernel

For $|u| \leq 1$:

$$K(u) = \frac{15}{16}(1-u^2)^2, \tag{45}$$
$$K'(u) = -\frac{15}{4}u(1-u^2), \tag{46}$$
$$K''(u) = \frac{15}{4}(3u^2-1). \tag{47}$$

The convolution for $|u| \leq 2$:

$$K_2(u) = \frac{5}{7} - \frac{15u^2}{14} + \frac{15u^4}{16} - \frac{15|u|^5}{32} + \frac{15|u|^7}{448} - \frac{5|u|^9}{3584}, \tag{48}$$
$$K_2'(u) = \frac{15u}{3584}\left(-3|u|^7 + 56|u|^5 - 560|u|^3 + 896u^2 - 512\right), \tag{49}$$
$$K_2''(u) = -\frac{45|u|^7}{448} + \frac{45|u|^5}{32} - \frac{75|u|^3}{8} + \frac{45u^2}{4} - \frac{15}{7}. \tag{50}$$

A.4 Triweight Kernel

For $|u| \leq 1$:

$$K(u) = \frac{35}{32}(1-u^2)^3, \tag{51}$$
$$K'(u) = -\frac{105}{16}u(1-u^2)^2, \tag{52}$$
$$K''(u) = -\frac{105}{16}(1-6u^2+5u^4). \tag{53}$$

The convolution for $|u| \leq 2$ (degree-13 polynomial):

$$\begin{aligned} K_2(u) = {} & \frac{350}{429} - \frac{35u^2}{22} + \frac{35u^4}{24} - \frac{35|u|^6}{32} + \frac{35|u|^7}{64} \\ & - \frac{35|u|^9}{768} + \frac{35|u|^{11}}{11264} - \frac{175|u|^{13}}{1757184}, \end{aligned} \tag{54}$$
$$K_2'(u) = \frac{35u}{135168}\left(-5|u|^{11} + 132|u|^9 - 1584|u|^7 + 14784|u|^5 - 25344u^4 + 22528u^2 - 12288\right), \tag{55}$$
$$K_2''(u) = -\frac{175|u|^{11}}{11264} + \frac{175|u|^9}{512} - \frac{105|u|^7}{32} + \frac{735|u|^5}{32} - \frac{525u^4}{16} + \frac{35u^2}{2} - \frac{35}{11}. \tag{56}$$

A.5 Cosine Kernel

For $|u| \leq 1$:

$$K(u) = \frac{\pi}{4} \cos\left(\frac{\pi u}{2}\right), \tag{57}$$

$$K'(u) = -\frac{\pi^2}{8} \sin\left(\frac{\pi u}{2}\right), \tag{58}$$

$$K''(u) = -\frac{\pi^3}{16} \cos\left(\frac{\pi u}{2}\right). \tag{59}$$

The convolution for $|u| \leq 2$:

$$K_2(u) = \frac{\pi}{32}\left[\pi(2-|u|)\cos\left(\frac{\pi|u|}{2}\right) + 2\sin\left(\frac{\pi|u|}{2}\right)\right], \tag{60}$$

$$K_2'(u) = -\text{sign}(u)\frac{\pi^3}{64}\sin\left(\frac{\pi|u|}{2}\right)(2-|u|), \tag{61}$$

$$K_2''(u) = \frac{\pi^3}{128}\left[\pi(|u|-2)\cos\left(\frac{\pi|u|}{2}\right) + 2\sin\left(\frac{\pi|u|}{2}\right)\right]. \tag{62}$$

A.6 NW Regression Weight Derivatives

For Nadaraya–Watson regression, the weight function $w_{ij} = K(u_{ij})/h$ requires derivatives with respect to h. Using $u = (x_i - x_j)/h$, we have $\partial u/\partial h = -u/h$.

For any kernel with K, K', K'':

$$w = \frac{K(u)}{h}, \tag{63}$$

$$w' = \frac{1}{h}\left[K'(u)\cdot\left(-\frac{u}{h}\right) - \frac{K(u)}{h}\right] = -\frac{1}{h^2}\left[uK'(u) + K(u)\right], \tag{64}$$

$$w'' = \frac{1}{h^3}\left[u^2K''(u) + 3uK'(u) + 2K(u)\right]. \tag{65}$$

For the Gaussian kernel, these simplify to:

$$w' = \frac{w(u^2-1)}{h}, \tag{66}$$

$$w'' = \frac{w(u^4-3u^2+1)}{h^2}. \tag{67}$$